

LABORATORY 02 – OS Setup, Shell, and Network Support Software

AUTORS

Andrés Felipe Chavarro Plazas

David Santiago Espinosa Rojas

TEACHER

Diego Alejandro Murcia Cespedes



COMPUTER NETWORKS

GROUP 01

Bogota, Colombia

February 9, 2025

OBJECTIVE

Learn to manage and use virtual network simulation environments in order to be able to apply said knowledge acquired in a real context. With the support of virtualization software and other software specialized in the control of the physical networks present in the laboratory, we will create scripts that support us in the use of various Unix operating systems and some other tools to use them in the right way.

MATERIALS & TOOLS

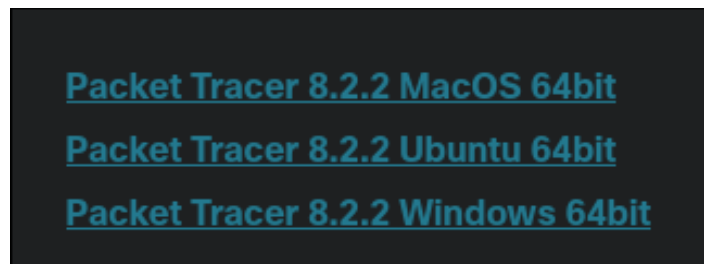
- Laboratory Computers
- Internet Access
- Virtualization Software
- Operating System Images
- *Packet Tracer*
- *Wireshark*

INTRODUCTION

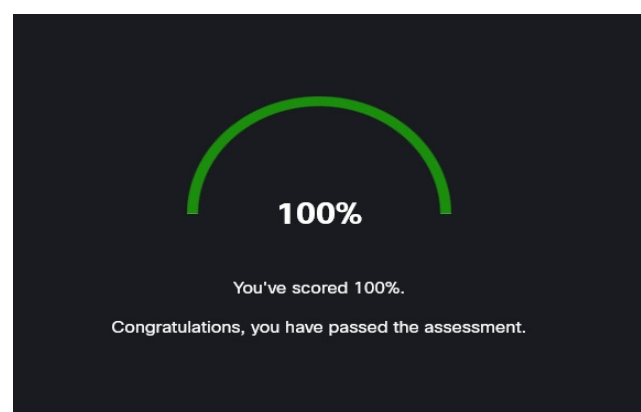
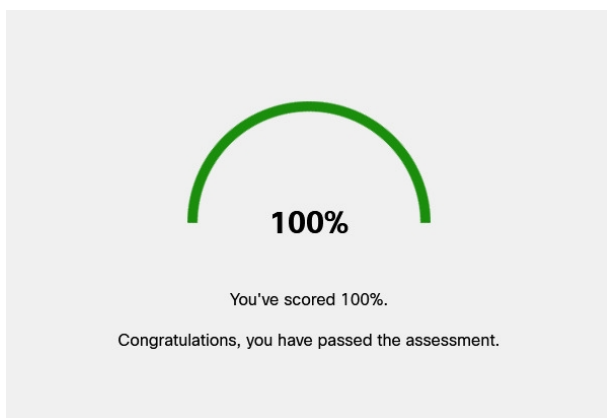
Nowadays, there are many ways to interact and use computer networks. However, to learn from them, multiple platforms and programs have been invented that simulate virtual networks and allow us to experience similarities with a real environment. These programs are aimed at those seeking to model processes on the network without the need to have the infrastructure to deploy a functional one. Once we acquire knowledge of the operation of networks, we use laboratories or data centers with the necessary infrastructure so that a network can be established. In them, we can use programs dedicated to network control, in which every possible aspect and configuration is controlled, that facilitates or improves the capacity and usefulness of the networks. On the other hand, using virtualization and, in some cases, real infrastructure, we can set up operating systems that will allow us to use the programs already mentioned and others more internal to the OS such as code editors. With them, scripts can be made that will help us in the task of managing networks in a more automated and controlled way.

GETTING TO KNOW PACKET TRACE

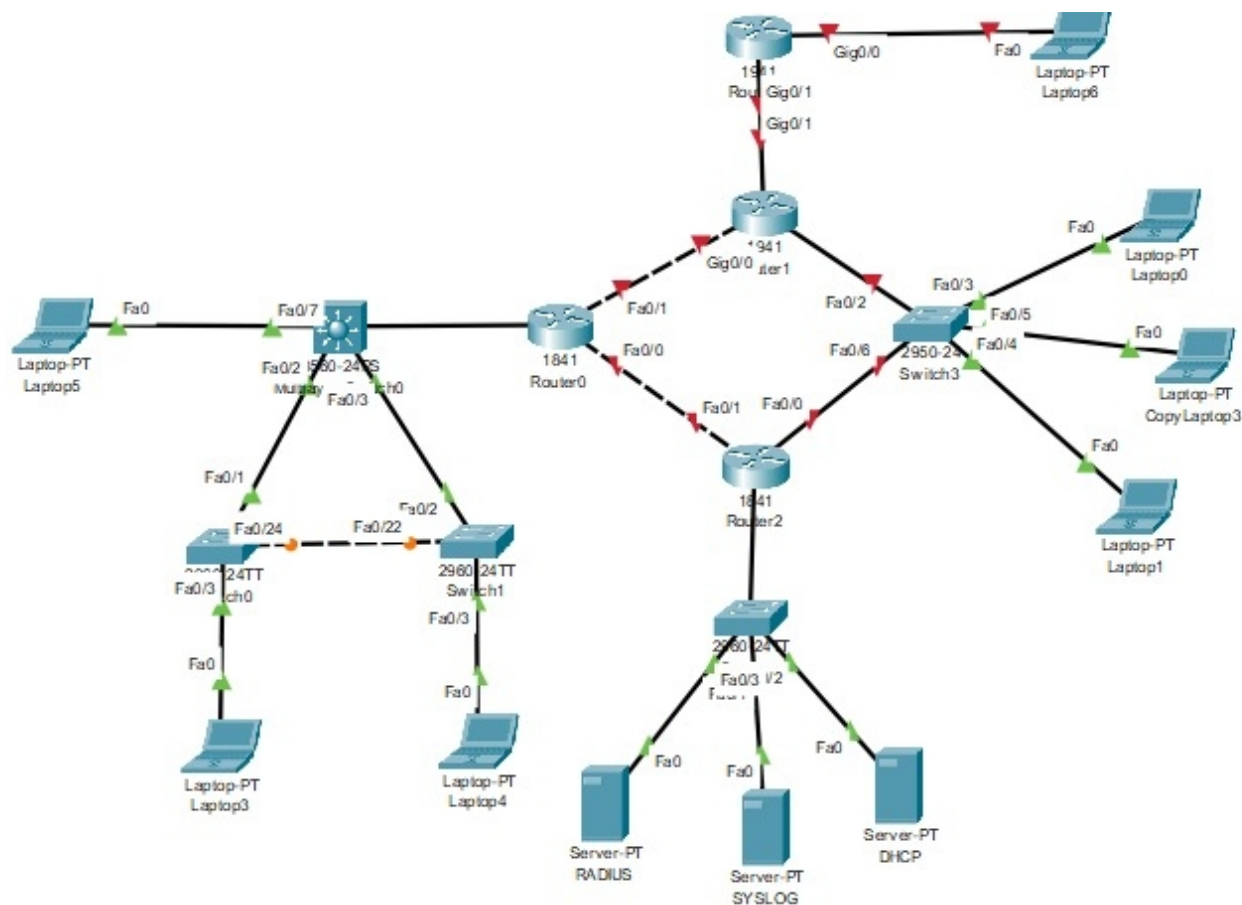
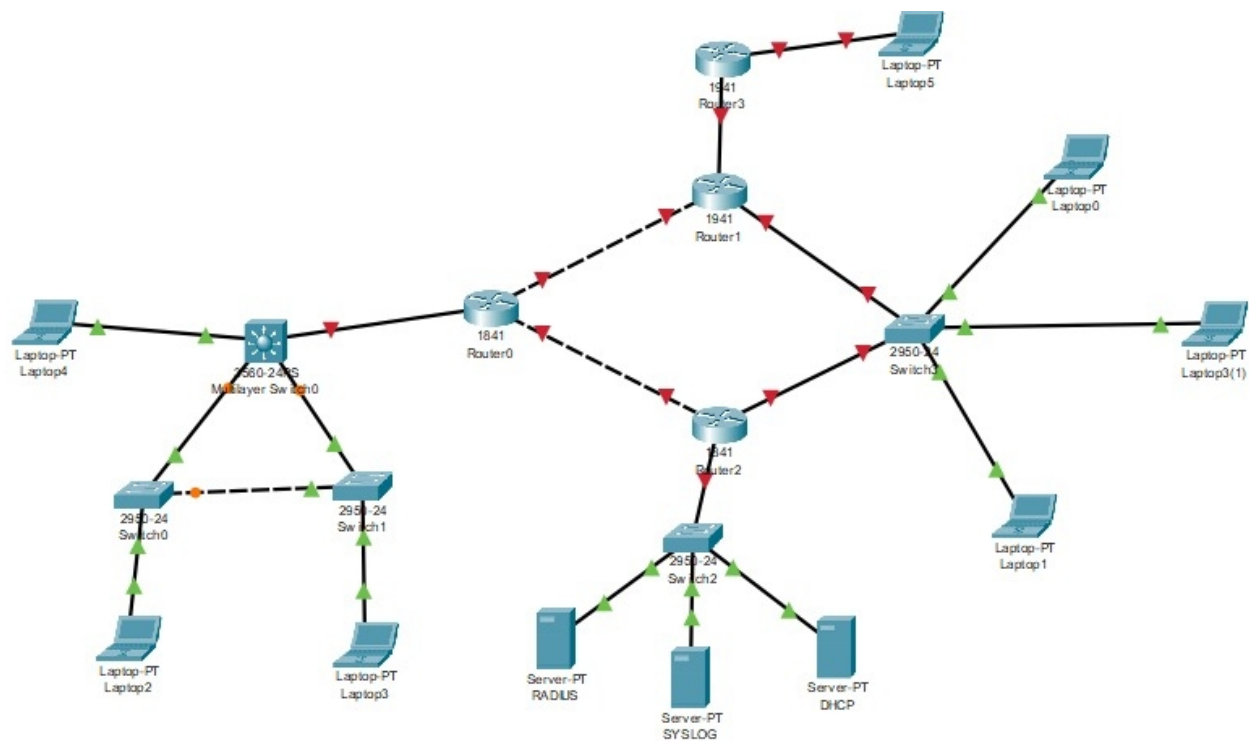
For this lab, we are going to use the specialist software *Packet Tracer*. This is given to us by the platform *CISCO* as a resource to beginners on this platform. There is a course called “*Getting Started with Cisco Packet Tracer*” that teaches us the basics of the program, with some other courses we can learn all about this software, from installation to its usefulness. Also, through the resources page (<https://www.netacad.com/resources/lab-downloads?courseLang=en-US>), we can obtain the program in its latest version (8.2.2) on different platforms.



As we already mentioned, the course is designed to learn from *Packet Tracer*. We completed the course, and made the video corresponding to the summary of the first chapters (link in the section corresponding to the videos at the end of the document). Additionally, we each take our corresponding quiz, as seen below (David’s exam is white and the dark one was made by Andres).



To corroborate that the course was useful to us, we are asked that each of the members make a replica of a specific diagram from the guide. These are the results taking into account the requirements given for the use of colors and shapes.



In these diagrams, we are forced to ignore certain characteristics of the program, since these are more advanced topics in the course. However, we can see that we were asked to differentiate the types of black wiring that were used to connect the devices. These cables are divided into two:

- **Solid black:** High-speed wired connections such as Ethernet, which are connected using switches, are also typical in LANs.
- **Dashed black:** Point-to-point serial connections are slower and used for long-distance links, ideal for WAN connections and use of protocols such as HDLC.

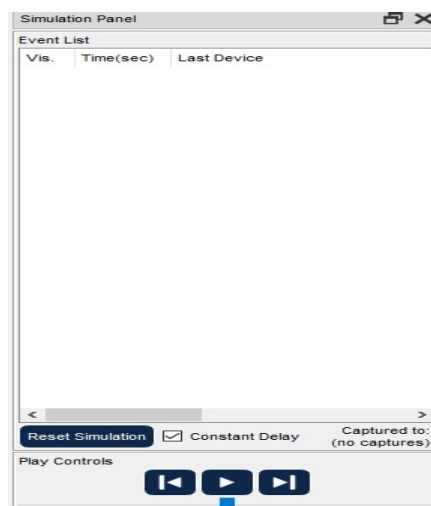
However, there are also other types of cables, in our case we had to use a serial cable as part of a small modification. This is intended to simulate a WAN network connection or environment, that is, these red cables are used to create a WAN network.

TRACKING MESSAGES WITH PACKER TRACER

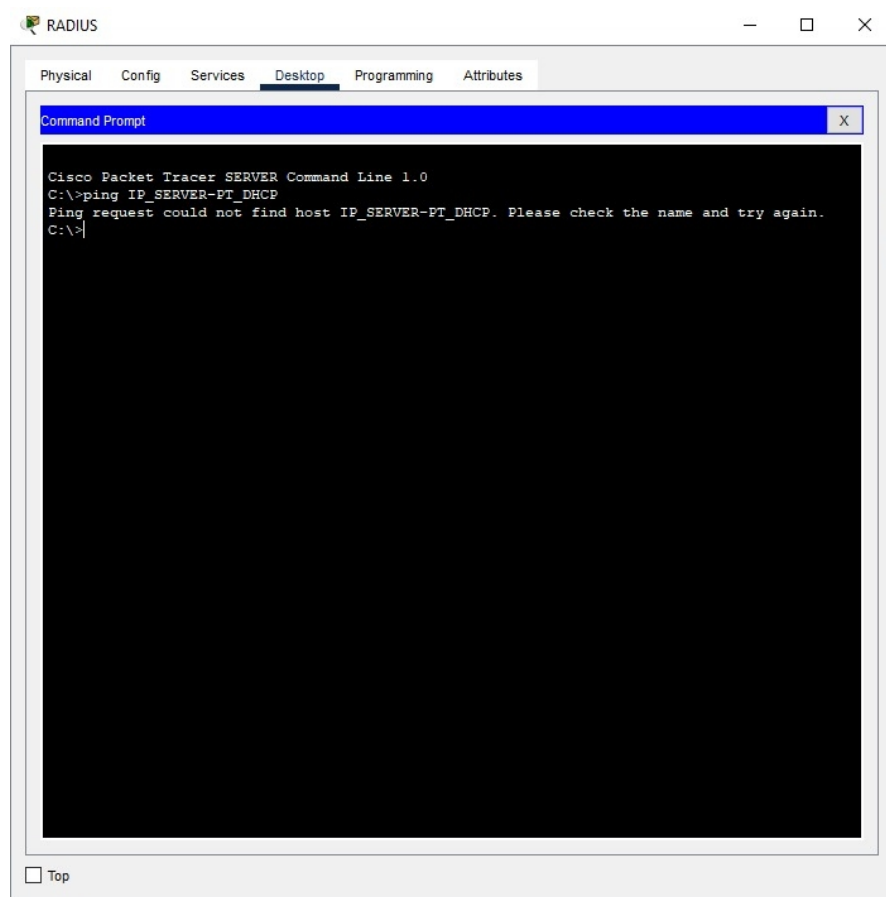
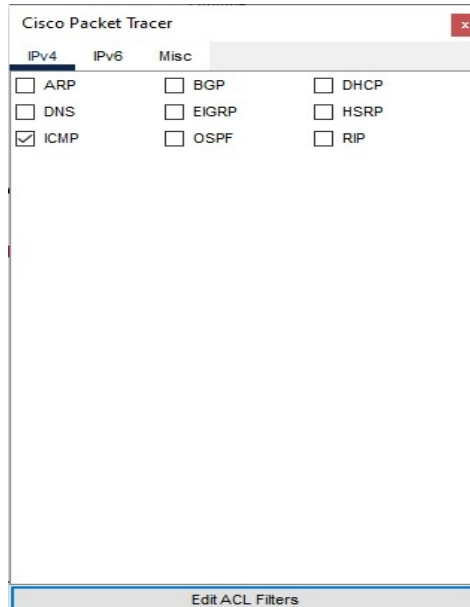
Based on the previous diagram, we are going to carry out a series of steps or actions that will allow us to see how the different elements that were used in the diagram behave. In addition, we will delve into certain tools that the software offers us with respect to network simulation and its subsequent analysis.

In this part, the ping was requested from the RADIUS server to the DHCP server, from that query 4 packets arose, 2 of which are first the request from radius and the movement through switch 2. That was the result process.

Based on the guide, the recommended steps were followed to evaluate the situation of the PDU's. As we already mentioned, we got a couple of packages that turned out correct and others that didn't, this can be explained by analyzing the event center and repeating the simulation.



In this new simulation, we apply the recommended ICMP filter, then we go to the radius configuration and configure it with the IP of the target server that was given, and we do the ping request. This way, we can obtain better results when performing the simulation. After following the guide, the two packages that came from the switch were obtained and arrived safely in the event queue.



ON THE REAL NETWORK

Now we are going to see exactly how the networks behave internally. For this, we are going to use a tool specialized in analyzing protocols and network traffic to diagnose and avoid insecurities or failures.

USING WIRESHARK

To start, we will need to download the *Wireshark* software from its official website (<https://www.wireshark.org/download.html>), it will only ask us to follow the necessary steps for its installation depending on the operating system. However, we use the previously configured computers in the network laboratory.

Once we have the tool, it is necessary to investigate and understand its operation in order to use it correctly. *Wireshark* is a network protocol analysis tool that allows you to capture, inspect and analyze data traffic in real time or from saved files, facilitating the diagnosis of network problems, security auditing such as detecting malicious activities, and detailed study of protocols. Using its graphical interface and advanced filters, users can examine individual packets, understand their structure and flow, and monitor network behavior, making it essential for system administrators, cybersecurity professionals, and enthusiasts seeking to understand the inner workings of digital communications. It is free, open source and compatible with multiple platforms.

Multiple devices can be connected to the network that could be analyzed with this software. Well, *Wireshark* analyzes the network flow through the network cards in each device. These are hardware devices that allow a computer to connect to a network, either by cable or wirelessly, allowing the sending and receiving of data through protocols such as TCP/IP. Additionally, they allow the operating system to consult data regarding the network from the same console, using commands such as ``ipconfig`` or ``ifconfig``.

Within Wireshark there are multiple ways to perform filters and provide better packet metrics as a result. Additionally, we can query specific domains and see how the device and server interact. For all this, an explanatory video is made, which is found in the video section.

Now that we understand how the software works, now we are going to use the tool. To do this, we had to analyze a query on the internet, in this case with the *SciELO Colombia* page. To begin, we open Wireshark and select the network card that the device is using, in our case Ethernet 4. Once open, the

system begins to show all the traffic and varies the data by color or based on what is displayed. Afterward, we open the target page in the browser and see how multiple network packets begin to appear in the analyzer.

The screenshot displays two windows side-by-side. The left window is a web browser showing the SciELO Colombia website. The page content includes information about the Pan-American Health Organisation (Colombia), the Universidad Nacional de Colombia, and the BIREME project. It mentions that the library forms part of a project being developed by FAPESP (Fundação de Amparo à Pesquisa do Estado de São Paulo) in collaboration with BIREME (Centro Latinoamericano y del Caribe de Información en Ciencias de la Salud). The FAPESP/BIREME project envisages developing a common methodology for preparing, storing, disseminating and evaluating scientific literature in electronic form. The website is designed for enabling access to SciELO's network of sites. Searches can be made of SciELO's existing collections covering the whole network or just a particular country. The site will be constantly updated both in form and content, according to the project's advancements.

The right window is Wireshark, showing a list of captured network packets. The packets are filtered by 'Apply a display filter -> <Ctrl-F>'. The list shows various protocols including LLMNR, DNS, ARP, and TCP. The selected packet is a Hypertext Transfer Protocol (HTTP) packet, which is a GET request to the SciELO Colombia website. The packet details show the request line: 'GET / HTTP/1.1'. The packet bytes pane shows the raw data of the packet.

As we can see, there is a lot of information of all kinds, so they ask us to filter the packages so that only those that begin with the word “Get” are shown. To do so, we have to place the following command `http.request.method == “get”` in the filters that we explained previously. Afterward, we just refresh the page and see how the packages were obtained. Something curious is that it only works with pages with the HTTP protocol, since HTTPS is encrypted and impossible to obtain with GET.

```

> Frame 12318: 752 bytes on wire (6016 bits), 752 bytes captured (6016 bits) on interface \Device\NPF_
> Ethernet II, Src: HP_6a:18:7b (30:13:8b:6a:18:7b), Dst: VMware_16:e6:c3 (00:0c:29:16:e6:c3)
> Internet Protocol Version 4, Src: 10.2.67.111, Dst: 168.176.28.57
> Transmission Control Protocol, Src Port: 56215, Dst Port: 80, Seq: 1, Ack: 1, Len: 698
▼ Hypertext Transfer Protocol
  > GET / HTTP/1.1\r\n
    Host: www.scielo.org.co\r\n
    Connection: keep-alive\r\n
    Upgrade-Insecure-Requests: 1\r\n
    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/132.0.0.0 Safari/537.36
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8
    Accept-Encoding: gzip, deflate\r\n
    Accept-Language: en-US,en;q=0.9\r\n
  > [truncated]Cookie: __utma=203976338.1344891441.1738869530.1738869530.1738869530.1; __utms=203976338.1
    \r\n
    [Full request URI: http://www.scielo.org.co/]
    [HTTP request 1/1]
    [Response in frame: 12632]

```

0000	00 0c 29 16 e6 c3 30 13 8b 6a 18 7b 08 00 45 00	..).0.j.{.E.
0010	02 e2 a2 8f 40 00 80 06 00 00 0a 02 43 6f a8 b0	...@...Co.
0020	1c 39 db 97 00 50 a6 10 3b 64 f7 80 0c 7b 50 18	.9...P.;d...{P.
0030	02 01 15 2f 00 00 47 45 54 20 2f 20 48 54 54 50	.../...GE T / HTTP
0040	2f 31 2e 31 0d 0a 48 6f 73 74 3a 20 77 77 77 2e	/1.1...Ho st: www.
0050	73 63 69 65 6c 6f 2e 6f 72 67 2e 63 6f 0d 0a 43	scielo.o rg.co.C
0060	6f 6e 6e 65 63 74 69 6f 6e 3a 20 6b 65 65 70 2d	onnectio n: keep-
0070	61 6c 69 76 65 0d 0a 55 70 67 72 61 64 65 2d 49	alive..U pgrade-I
0080	6e 73 65 63 75 72 65 2d 52 65 71 75 65 73 74 73	nsecure- Requests
0090	3a 20 31 0d 0a 55 73 65 72 2d 41 67 65 6e 74 3a	: 1..Use r-Agent:
00a0	20 4d 6f 7a 69 6c 6c 61 2f 35 2e 30 20 28 57 69	Mozilla /5.0 (Wi
00b0	6e 64 6f 77 73 20 4e 54 20 31 30 2e 30 3b 20 57	ndows NT 10.0; W
00c0	69 6e 36 34 3b 20 78 36 34 29 20 41 70 70 6c 65	in64; x6 4) Apple
00d0	57 65 62 4b 69 74 2f 35 33 37 2e 33 36 20 28 4b	WebKit/5 37.36 (K
00e0	48 54 4d 4c 2c 20 6c 69 6b 65 20 47 65 63 6b 6f	HTML, li ke Gecko
00f0	29 20 43 68 72 6f 6d 65 2f 31 33 32 2e 30 2e 30	) Chrome /132.0.0
0100	2e 30 20 53 61 66 61 72 69 2f 35 33 37 2e 33 36	.0 Safar i/537.36
0110	20 45 64 67 2f 31 33 32 2e 30 2e 30 2e 30 0d 0a	Edg/132 .0.0.0..
0120	41 63 63 65 70 74 3a 20 74 65 78 74 2f 68 74 6d	Accept: text/htm
0130	6c 2c 61 70 70 6c 69 63 61 74 69 6f 6e 2f 78 68	l,applic ation/xh
0140	74 6d 6c 2b 78 6d 6c 2c 61 70 70 6c 69 63 61 74	tml+xml, applicat
0150	69 6f 6e 2f 78 6d 6c 3b 71 3d 30 2e 39 2c 69 6d	ion/xml; q=0.9,im
0160	61 67 65 2f 61 76 69 66 2c 69 6d 61 67 65 2f 77	age/avif ,image/w
0170	65 62 70 2c 69 6d 61 67 65 2f 61 70 6e 67 2c 2a	ebp,imag e/apng,*

No: 12318 · Time: 141.925040 · Source: 10.2.67.111 · Destination: 168.176.28.57 · Protocol: HTTP · Length: 752 · Info: GET / HTTP/1.1

☒ Show packet bytes

Close

Help

The image shows a web browser window displaying the SciELO Colombia website. The page title is "Scientific Electronic Library Online". The main content area includes a navigation menu with links like "serial browsing", "article browsing", and "reports". Below the menu, there is a section titled "SciELO" which describes the library as a virtual library for Latin-America, the Caribbean, Spain and Portugal. It mentions that it consists of a network operating in two fields: Regional collections (SciELO Brazil, SciELO Chile, SciELO Cuba, SciELO Colombia, etc.) and Thematic areas (SciELO public health). The page also mentions that the library forms part of a project being developed by FAPESP (Fundação de Amparo à Pesquisa do Estado de São Paulo) in collaboration with BIREME (Centro Latinoamericano y del Caribe de Información en Ciencias de la Salud). The page footer includes a "help" section with links to "about this site" and "help about this site".

Overlaid on the right side of the browser window is a Wireshark packet capture window. The top of the Wireshark window shows the "Packet List" pane with a table of captured packets. The selected packet is packet 3121, which is an HTTP GET request to the SciELO Colombia website. The packet details pane shows the structure of the HTTP request, including the "Hypertext Transfer Protocol" section. The packet bytes pane shows the raw data of the packet in hexadecimal and ASCII format.

Now, we are going to analyze another 3 different websites to continue exploring Wireshark. To do this, we will use different filters that will allow us to have greater clarity about what is being presented. Some of these were: `tls.handshake.extensions_server_name contains "<Web>"` to see all packages from that source; `tcp.port==443` to show all HTTPS request; `ip.addr == <ip of web>` to see all about the ip from the page we are looking, this is posible doing `ping <page>` in the terminal. However, there is a more explanatory video for the following images, that video is in the videos section.

For the following pages, we reviewed what packages they gave us with the filters already mentioned. Each one had different behaviors.

The first page is *Archlinux.org*, on this we did not see strange or unusual behavior. Likewise, there was nothing inside the packages either.

Arch Linux website and Wireshark packet capture analysis.

Arch Linux Website:

- A simple, lightweight distribution**
- Recent Updates (more):** pwndbg 2025.01.20-2, pyrsid6 6.8.2.1-1, soc-scheds 1.0.9-1, pygitb 6.8.1-1, zamshin 24.12.2-1, yakuake 24.12.2-1, umbrallo 24.12.2-1, lokodon 24.12.2-1, telly-skrout 24.12.2-1, sweeper 24.12.2-1, svppart 24.12.2-1, step 24.12.2-1, spectacle 24.12.2-1, skidashk 24.12.2-1, skanpage 24.12.2-1.
- Documentation:** Wiki, Manual Pages, Installation Guide.
- Community:** Mailing Lists, IRC Channels, Planet Arch, International Communities.
- Support:** Donors, Products via Unmaintainers, T-shirts via FreeBeer, T-shirts via HELLOTUX.
- Tools:** Mirrors/Updater, Mirror List, Mirror Status.
- Development:**

Wireshark Packet Capture:

- Filter:** `ip.addr == 95.217.163.246 || tcp.port == 443`
- Packet 111:** 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \Device\NPF_{D48...}
- Ethernet II:** Src: VMware_67:6f:02 (00:0c:29:67:6f:02), Dst: HP_6a:18:7b (30:13:8b:6a:18:7b)
- Internet Protocol Version 4:** Src: 20.42.65.91, Dst: 10.2.67.111
- TCP:** Src Port: 443, Dst Port: 56410, Seq: 7757, Ack: 12732, Len: 0

Wireshark - Packet 79113 - Ethernet 4

> Frame 79113: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface \Device\NPF_{D48...}

> Ethernet II, Src: VMware_67:6f:02 (00:0c:29:67:6f:02), Dst: HP_6a:18:7b (30:13:8b:6a:18:7b)

> Internet Protocol Version 4, Src: 20.42.65.91, Dst: 10.2.67.111

> 0100 = Version: 4

> 0101 = Header Length: 20 bytes (5)

> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)

> Total Length: 40

> Identification: 0x82bd (33469)

> 010. = Flags: 0x2, Don't fragment

> ...0 0000 0000 0000 = Fragment Offset: 0

> Time to Live: 254

> Protocol: TCP (6)

> Header Checksum: 0x571c [validation disabled]

> [Header checksum status: Unverified]

> Source Address: 20.42.65.91

> Destination Address: 10.2.67.111

> Transmission Control Protocol, Src Port: 443, Dst Port: 56410, Seq: 7757, Ack: 12732, Len: 0

> Source Port: 443

0000 30 13 8b 6a 18 7b 00 0c 29 67 6f 02 08 00 45 00 00 j { . } go . . E .

0010 00 28 82 bd 40 00 fe 06 57 1c 14 2a 41 5b 0a 02 . (. @ . . . W . * A [. .

0020 43 6f 01 bb dc 5a 28 30 7e 78 e8 0e 35 8e 50 10 Co . . . Z (0 ~ x . . 5 . P .

0030 10 00 5a 83 00 00 00 00 00 00 00 00 00 00 00 00 - Z

No.: 79113 · Time: 1034.859875 · Source: 20.42.65.91 · Destination: 10.2.67.111 · ... Info: [TCP Keep-Alive ACK] 443 → 56410 [ACK] Seq=7757 Ack=12732 Win=262144 Len=0

☒ Show packet bytes

Close Help

The next page is "github.com", on this we found something curious with some packages that were in an encrypted state. From what could be found out, GitHub encrypts some things on its website to keep computer security in its best state, thus avoiding any vulnerability.

The screenshot shows a web browser window displaying the GitHub homepage. The page features the GitHub logo, a navigation menu, and a large heading "Build and ship software on a single, collaborative platform". Below the heading, there is a form to "Enter your email" and buttons for "Sign up for GitHub" and "Try GitHub Copilot".

Overlaid on the right side of the browser window is a Wireshark packet capture window. The top pane shows a list of captured packets, with the selected packet being a TCP segment from 10.2.67.112 to 20.42.73.27. The bottom pane shows the details of the selected packet, including the Ethernet II header, Internet Protocol Version 4 header, and Transmission Control Protocol header. The packet is identified as a SYN packet (Seq=1, Ack=1, Len=0).

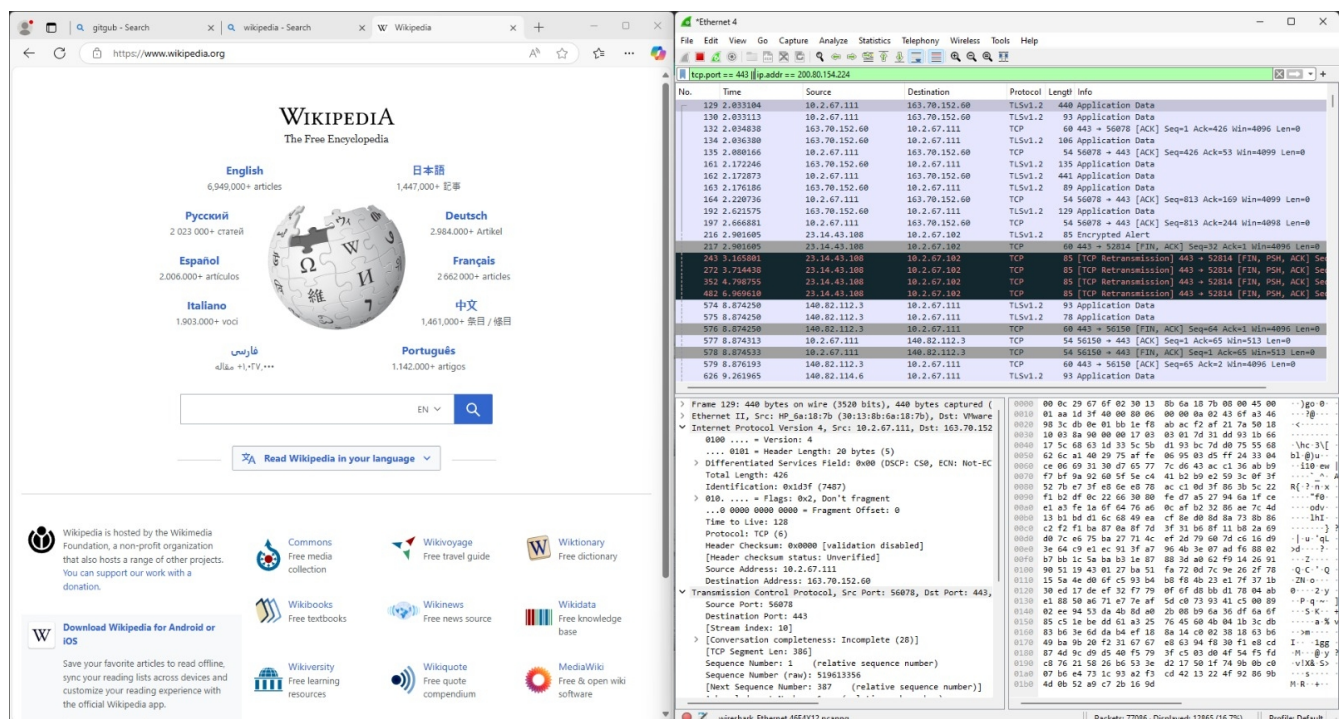
The screenshot shows a Wireshark packet capture window. The top pane shows a list of captured packets, with the selected packet being a SYN packet from 10.2.67.112 to 20.42.73.27. The bottom pane shows the details of the selected packet, including the Ethernet II header, Internet Protocol Version 4 header, and Transmission Control Protocol header. The packet is identified as a SYN packet (Seq=1, Ack=1, Len=0).

The packet details pane shows the following information:

- Frame 204: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface \Device\NPF_{D48BA77...}
- Ethernet II, Src: VMware_67:6f:02 (00:0c:29:67:6f:02), Dst: VMware_67:6f:02 (00:0c:29:67:6f:02)
- Destination: VMware_67:6f:02 (00:0c:29:67:6f:02)
- Source: HP_6a:19:53 (30:13:8b:6a:19:53)
- Type: IPv4 (0x0800)
- Internet Protocol Version 4, Src: 10.2.67.112, Dst: 20.42.73.27
- Transmission Control Protocol, Src Port: 51246, Dst Port: 443, Seq: 1, Ack: 1, Len: 0

The packet bytes pane shows the raw data of the packet in hexadecimal and ASCII format.

The page was "*wikipedia.com*", in this we only found some packages that we could not determine their nature, since the color they had was the same as the encryption color but they were not encrypted. From what was investigated, it may be due to errors within the browser that cause some packages to have alterations.



NETWORK CARDS

As we already mentioned, network cards are the element that allows us to communicate devices with networks. In this part, we are only going to use different devices and virtual machines to see all the information that is requested of us based on the networks and the network card. For devices with any Unix installed, we used the commands *ip a* or *ifconfig* for display the network carts and their information, *lspci | grep -i net* for display the exact model of the controller, *iwconfig* for display the

info of the net if the device is wireless like a laptop. In the case of Windows, we only use the command 'ipconf' for display the information. In this way, we obtained the following results:

- Windows GUI virtual machine:

```
Ethernet adapter Ethernet 4:

Connection-specific DNS Suffix . : is.escuelaing.edu.co
Description . . . . . : Intel(R) Ethernet Connection (17) I219-LM
Physical Address. . . . . : 30-13-8B-6A-18-7B
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::815e:cc59:8af6:dab7%17(Preferred)
IPv4 Address. . . . . : 10.2.67.111(Preferred)
Subnet Mask . . . . . : 255.255.0.0
Lease Obtained. . . . . : Thursday, February 6, 2025 10:15:36 AM
Lease Expires . . . . . : Friday, February 7, 2025 1:38:52 AM
Default Gateway . . . . . : 10.2.65.1
                             10.2.65.3
DHCP Server . . . . . : 10.2.65.3
DHCPv6 IAID . . . . . : 422581131
DHCPv6 Client DUID. . . . . : 00-01-00-01-2E-EA-50-BB-18-60-24-DE-F2-91
DNS Servers . . . . . : 10.2.65.2
                             10.2.65.61
                             10.2.65.62
NetBIOS over Tcpip. . . . . : Enabled
```

- NetBSD virtual machine

```
root@maquina:~# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.2.77.30 netmask 255.255.0.0 broadcast 10.2.255.255
    inet6 fe80::20c:29ff:fe97:5f84 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:97:5f:84 txqueuelen 1000 (Ethernet)
    RX packets 778 bytes 54548 (53.2 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 5 bytes 446 (446.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@maquina:~# t_
```

```
root@maquina:~# ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP mode DEFAULT group default qlen 1000
    link/ether 00:0c:29:97:5f:84 brd ff:ff:ff:ff:ff:ff
root@maquina:~#
```



```

root@maquina:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:0c:29:97:5f:84 brd ff:ff:ff:ff:ff:ff
    inet 10.2.77.30/16 brd 10.2.255.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe97:5f84/64 scope link
        valid_lft forever preferred_lft forever
root@maquina:~# _

```

- Andrés's laptop with Arch Linux

```

> iwconfig
lo          no wireless extensions.

wlan0      IEEE 802.11  ESSID:"labinfo_56"
          Mode:Managed  Frequency:5.2 GHz  Access Point: 04:42:1A:51:5D:E4
          Bit Rate=117 Mb/s   Tx-Power=30 dBm
          Retry short limit:7   RTS thr:off   Fragment thr:off
          Power Management:on
          Link Quality=52/70  Signal level=-58 dBm
          Rx invalid nwid:0  Rx invalid crypt:0  Rx invalid frag:0
          Tx excessive retries:0  Invalid misc:25  Missed beacon:0

```

```

> lspci | grep -i net

```

```

01:00.0 Network controller: Qualcomm Atheros QCA6174 802.11ac Wireless Network Adapter (rev 32)

```



```
> ip a
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 14:5a:fc:38:60:b3 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.253/24 brd 192.168.50.255 scope global dynamic noprefixroute wlan0
        valid_lft 83645sec preferred_lft 83645sec
    inet6 fe80::f480:2938:eb71:d048/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

- David's laptop with Windows

Adaptador de LAN inalámbrica Wi-Fi 2:

```
Sufijo DNS específico para la conexión. . . :
Descripción . . . . . : Intel(R) Wi-Fi 6 AX203
Dirección física. . . . . : A0-B3-39-5E-A5-6F
DHCP habilitado . . . . . : sí
Configuración automática habilitada . . . : sí
Vínculo: dirección IPv6 local. . . : fe80::1a0e:1027:eec:969c%14(Preferido)
Dirección IPv4. . . . . : 192.168.50.186(Preferido)
Máscara de subred . . . . . : 255.255.255.0
Concesión obtenida. . . . . : jueves, 6 de febrero de 2025 2:13:34 p. m.
La concesión expira . . . . . : viernes, 7 de febrero de 2025 2:13:33 p. m.
Puerta de enlace predeterminada . . . . . : 192.168.50.1
Servidor DHCP . . . . . : 192.168.50.1
IAID DHCPv6 . . . . . : 281548110
DUID de cliente DHCPv6. . . . . : 00-01-00-01-2E-D9-15-10-A0-B3-39-5E-A5-6F
Servidores DNS. . . . . : 192.168.50.1
NetBIOS sobre TCP/IP. . . . . : habilitado
```

BASE SOFTWARE

In real environments where the infrastructure is already set up and configured with some OS, it is common to need certain knowledge of internal and general Unix tools. In this case, we are going to see how some code editors work to be able to create and edit scripts that automate certain processes. These processes are usually repetitive tasks that require manual execution to be completed. On the other hand, we are going to see how to use the network to carry out data transfers and machine interconnections, in addition to reviewing the log to be able to check the internal behavior of the system.

SHELL PROGRAMING

As we already saw, the use of tools designed to analyze a network, is useful to understand its internal functioning. Also, use software inside the OS's that allow run some functionalities like scripst, would helps to improve the workflow. For this reason, many operating systems allow the execution of scripts that facilitate tasks related to networks by automating within them. In this case, we first have to understand how to create them and how they work within the system. To do this, we are going to create multiple scripts that will allow us to acquire the necessary knowledge for future applications in the following laboratories (*to see the scripts, see lab attachments. It's a lot of code to put in this document and make it comfortable.*).

- **Command `ls`:** in this script, a parameter will be received with the path of a directory, it will validate its existence and subsequently give multiple options to the user to organize or filter the content of the given directory. To do this, there is a menu that activates multiple variables and conventions of the `ls` command.
- **File Search and Viewing Commands:** This script only serves to analyze and find certain words within files or the latter in the given directories. To do this, the ***find*** command is used and variants or conjunctions with others
- **Log File Review:** This script only works to display a fragment of the system logs. However, you have to understand what logs are and what to do. Log files record important events, actions and messages generated by the operating system and applications, being essential for monitoring, diagnosis and auditing. The main types of logs include system, application, and security logs, which are present in most operating systems. Syslog is a standard that defines

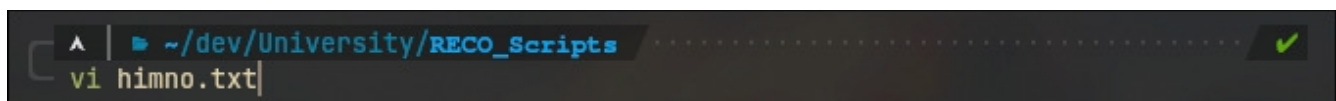
how logs are generated, transmitted and stored, facilitating their centralized management. Many operating systems follow this standard, although their implementation may vary depending on the configuration and tools used.

- **User Creation:** In this last script, we will only automatically create users and they will have their permissions, groups, roles and parameters defined. In this way, we save a lot of time on these laborious tasks.

VI EDITOR IN LINUX/UNIX

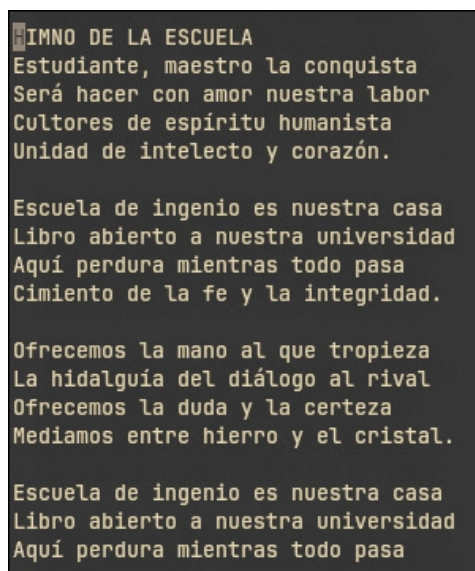
In Unix operating systems there are various editors, *Vi* is one of the oldest, but functional there are, it is very light and practically does not use system resources. For this exercise, we are going to follow the following steps indicated in the guide:

1. To create a file with *Vi*, in the directory where we want it and in the console, we put the command ``vi name.txt``. Where name could be whatever you want to give to the file, and an extension depends on the type of file.



```
^ | ~ /dev/University/RECO_Scripts |  
vi himno.txt
```

2. *Vi* works through a system of modes, wherein one you can insert code, in another select, and the last one to navigate the file. In this case, we press the ``i`` key to be able to insert the code and move with ``j`` to down, ``k`` to up, ``l`` to right, ``h`` to left. In our case, we pasted the university anthem and made the respective separation of paragraphs.



```
IMNO DE LA ESCUELA  
Estudiante, maestro la conquista  
Será hacer con amor nuestra labor  
Cultores de espíritu humanista  
Unidad de intelecto y corazón.  
  
Escuela de ingenio es nuestra casa  
Libro abierto a nuestra universidad  
Aquí perdura mientras todo pasa  
Cimiento de la fe y la integridad.  
  
Ofrecemos la mano al que tropieza  
La hidalguía del diálogo al rival  
Ofrecemos la duda y la certeza  
Mediamos entre hierro y el cristal.  
  
Escuela de ingenio es nuestra casa  
Libro abierto a nuestra universidad  
Aquí perdura mientras todo pasa
```

3. In *Vi*, we can press the key `;` which allows us to activate a command bar. This is used to execute the command we want according to what *Vi* has by default. In this case, we use the command `w` to save the file. If it had not been saved, the file does not remain in the system or does not maintain the changes made.

```
Escuela de ingenio es nuestra casa
Libro abierto a nuestra universidad
Aquí perdura mientras todo pasa
Cimiento de la fe y la integridad.
"himno.txt" 20 lines, 570 characters
```

4. To change all the vowels "a" with the requested symbol "-", we have to use the command bar and enter the following `2,5s/a/-/g`. This command is made up of: `2,5s/` indicates the lines that make up the first paragraph and that will be replaced; `a/-` indicates the change we want to make; `g` indicates what will be done on each line indicated.

```
Estudi-nte, m-estro l- conquist-
Será h-cer con -mor nustr- l-bor
Cultores de espíritu hum-nist-
Unid-d de intelecto y cor-zón.
```

5. To change all the strings "al" to the requested "##", we have to use the command bar and enter the following `%s|<al>/##/g`. This command is made up of; `%s/` indicates the lines that make up the file and that will be replaced; `%s|<al>/##` indicates the change we want to make; `g` indicates what will be done on each line indicated. In this case, only two strings were changed.

```
Ofrecemos la mano ## que tropieza
La hidalguía del diálogo ## rival
Ofrecemos la duda y la certeza
Mediamos entre hierro y el cristal.
```

6. In *Vi*, the `d` command is used to perform any type of elimination. In this case, the command to eliminate the word where we are is `dw`. Obviously, there are many more combinations to eliminate depending on what we need.
7. To delete the last lines we use the command `:.-3,\$d`, this is composed of: `.-3,` indicates that it takes the last 3 lines; `\$d` indicates that it deletes all the lines but is conditioned by the above.

```
Estudi-nte, m-estro l- conquist-  
Será h-cer con -mor nustr- l-bor  
Cultores de espíritu hum-nist-  
Unid-d de intelecto y cor-zón.
```

```
Escuela de ingenio es nuestra casa  
Libro abierto a nuestra universidad  
Aquí perdura mientras todo pasa  
Cimiento de la fe y la integridad.
```

```
Ofrecemos la mano ## que tropieza  
La hidalguía del diálogo ## rival  
Ofrecemos la duda y la certeza  
Mediamos entre hierro y el cristal.
```

```
█
```

```
~
```

```
~
```

```
~
```

```
~
```

```
4 lines
```

8. To return the changes made, we use the ``u`` command, this can be applied only to undo the last move or command, if we use it again, the original change will come back.
9. To delete the last lines we use the command ``$s/*^U&/``, this is composed of: ``$s/`` indicates that you take the last line and replace; ``.*`` indicates to change all characters; ``^U&/`` indicates to put all capital letters.

```
Escuela de ingenio es nuestra casa  
Libro abierto a nuestra universidad  
Aquí perdura mientras todo pasa  
CIMIENTO DE LA FE Y LA INTEGRIDAD.  
:$s/*^U&/
```

10. We position ourselves on the first of the last two lines of the second paragraph, then we write the command ``y2y``, where ``y`` indicates that we want to copy something and ``2y`` copies two lines and confirms, to paste them at the end, we place ``G`` which moves us to the last line, and we paste with ``p``.

```
Escuela de ingenio es nuestra casa  
Libro abierto a nuestra universidad  
Aquí perdura mientras todo pasa  
CIMIENTO DE LA FE Y LA INTEGRIDAD.  
Aquí perdura mientras todo pasa  
Cimiento de la fe y la integridad.  
"himno.txt"█
```


11. To find the word "*Escuela*", we simply enter the command `/Escuela` and with the `n` key we can vary between all occurrences of the word.
12. If we want to go from anywhere to line 5, we simply have to write the line as a command, in this case `5`.
13. To exit and save the file, we simply enter the `wq` command.

```
HIMNO DE LA ESCUELA
Estudi-nte, m-estro l- conquist-
Será h-cer con -mor nustr- l-bor
Cultores de espíritu hum-nist-
Unid-d de intelecto y cor-zón.
```

```
HIMNO DE LA ESCUELA
Estudi-nte, m-estro l- conquist-
Será h-cer con -mor nustr- l-bor
Cultores de espíritu hum-nist-
d de intelecto y cor-zón.
```

14. We open the command and in the first line we place the command `d5d`, where we indicate that we want to delete and the number of lines.

```
Escuela de ingenio es nuestra casa
Libro abierto a nuestra universidad
Aquí perdura mientras todo pasa
Cimiento de la fe y la integridad.

Ofrecemos la mano ## que tropieza
La hidalguía del diálogo ## rival
Ofrecemos la duda y la certeza
Mediamos entre hierro y el cristal.

Escuela de ingenio es nuestra casa
Libro abierto a nuestra universidad
Aquí perdura mientras todo pasa
CIMIENTO DE LA FE Y LA INTEGRIDAD.
Aquí perdura mientras todo pasa
Cimiento de la fe y la integridad.
~
~
```

15. To exit without saving, enter the exit command but with the force symbol, in this case the command `q!` would remain.

The process does not vary depending on the operating system, since the Vi editor uses its own commands and special keys to function. That's why we developed this part on a personal computer and then checked it again on a machine when we imported the lab files.

VIRTUAL MACHINE DEPLOYMENT

In this part, we did not understand very well what was being asked of us or how we could carry out the communication check. After some time investigating, we found that virtual machines have some conditions and/or parameters that have to be met in order for them to communicate. For example, authentication by SSH or FTP methods, internal services with open ports, firewall configurations on each machine and network connectivity. In the last mentioned, we are told that each machine must be able to connect to the network and access each other. To do this, the most common method is to perform a ping between the two and they should be able to communicate. This is because both are under the same network but with different IP's, allowing the machines to interact with each other because they are managed on the same network. En el siguiente ejemplo, usamos la maquina de *Slackware* con IP de `10.2.77.27` y la de *NetBSD* con la IP `10.2.77.26`.

- **Slackware:**

```
valid_tty forever preferred_tty forever
root@slack:/var/compartir# ping 10.2.77.26
PING 10.2.77.26 (10.2.77.26) 56(84) bytes of data.
64 bytes from 10.2.77.26: icmp_seq=1 ttl=255 time=3.67 ms
64 bytes from 10.2.77.26: icmp_seq=2 ttl=255 time=1.75 ms
64 bytes from 10.2.77.26: icmp_seq=3 ttl=255 time=1.89 ms
64 bytes from 10.2.77.26: icmp_seq=4 ttl=255 time=2.72 ms
64 bytes from 10.2.77.26: icmp_seq=5 ttl=255 time=2.38 ms
64 bytes from 10.2.77.26: icmp_seq=6 ttl=255 time=2.60 ms
64 bytes from 10.2.77.26: icmp_seq=7 ttl=255 time=1.72 ms
^C
```

- NetBSD:

```

enabled=0
ec_capabilities=7<ULAN_MTU,ULAN_HWTAGGING,JUMBO_MTU>
ec_enabled=2<ULAN_HWTAGGING>
address: 00:0c:29:02:53:c2
media: Ethernet autoselect (1000baseT full-duplex,master)
status: active
inet 10.2.77.26/16 broadcast 10.2.255.255 flags 0x0
inet6 fe80::20c:29ff:fe02:53c2%wm0/64 flags 0x0 scopeid 0x1
lo0: flags=0x8049<UP,LOOPBACK,RUNNING,MULTICAST> mtu 33176
inet 127.0.0.1/8 flags 0x0
inet6 ::1/128 flags 0x20<NODAD>
inet6 fe80::1%lo0/64 flags 0x0 scopeid 0x2
net# ping 10.2.77.27
PING 10.2.77.27 (10.2.77.27): 56 data bytes
64 bytes from 10.2.77.27: icmp_seq=0 ttl=64 time=1.461126 ms
64 bytes from 10.2.77.27: icmp_seq=1 ttl=64 time=1.771681 ms
64 bytes from 10.2.77.27: icmp_seq=2 ttl=64 time=1.813514 ms
64 bytes from 10.2.77.27: icmp_seq=3 ttl=64 time=2.701789 ms
64 bytes from 10.2.77.27: icmp_seq=4 ttl=64 time=1.563314 ms
64 bytes from 10.2.77.27: icmp_seq=5 ttl=64 time=2.248394 ms
^C
----10.2.77.27 PING Statistics----
6 packets transmitted, 6 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 1.461126/1.926636/2.701789/0.466883 ms
net# █

```

As we could see, the machines were capable of doing the network check with the IP of the other machine. In this way, we consider the relationship between them to be valid and proven.

FILE SHARING

To begin with, Samba is a package or software that uses a protocol called "SMB" and is used by multiple devices as resources on the network. This is used to share files and Samba is responsible for creating interconnectivity between multiple devices.

First of all, we have to install it. To do this, in Windows we download the *.exe* and follow the installation steps. For NetBSD, we activate the binary packages and then download the software package with *'pkg install samba'*. In Slackware, we use *'slackpkg install samba'*. Now, we will show how the configuration is in each case.

- NetBSD:

When Samba is installed, some of its dependencies are not installed, in this case mnbd, smbd and winbindd. However, we install them in the same way as the other packages.

Once everything is downloaded, we have to activate the samba service on the system, we can do it using the `service samba start` command.

```
net# service samba start
Starting nmbd.
Starting winbindd.
Starting smbd.
net# █
```

In some cases, it is advisable to start these services directly from the boot file. To do this, we place certain commands in the `/etc/rc.conf` file.

```
smbd=YES
nmbd=YES
winbindd=YES
samba=YES
```

However, we noticed that the default configuration was practically empty, so we placed a custom one in the file `/usr/pkg/etc/samba/smb.conf`. Unlike other distributions, in this one we have to balance each aspect of the server for some reason that we do not know, leaving the following result for the basic communication of the machines

```
[global]
workgroup = WORKGROUP
security = user map
to guest = Bad User
[homes] comment = Home irectories
browseable = no
writable = yes
comment = Home Directories
[printers] comment = All Printers
path = /var/spool/samba
browseable = no
guest ok = yes
printable = yes
[print$]
comment = Printer Drivers
path = /usr/pkg/share/cups/drivers
browseable = yes
guest ok = no
read only = yes
write list = @admin root
~
~
~
~
"/usr/pkg/etc/smb.conf" 28L, 424B
```

18, 14

All

Now with this configuration, we can start the package. However, it is necessary to create a Samba user that allows us to access other servers. To do this, it is necessary to create the user

previously, since for some reason it does not allow the creation of system users parallel to Samba. Once the user has been created (we previously saw how to do it, so we skip the explanation), we proceed to write the command `'smbpasswd -a -U simba'`, where we will be asked for a password for this one that will be more relevant later.

- **Slackware:**

To configure Slackware, we follow the same process as before, starting by installing the package with the `'slackpkg install samba'` command. In this case, the configuration is generated automatically in the directory `'/etc/samba/smb.conf'` and gives us the freedom to modify it to personal taste.

```
GNU nano 6.0 /etc/samba/smb.conf
[global]

# workgroup = NT-Domain-Name or Workgroup-Name, eg: LINUX2
workgroup = slack
security = user

# server string is the equivalent of the NT Description field
server string = Samba Server

# Server role. Defines in which mode Samba will operate. Possible
# values are "standalone server", "member server", "classic primary
# domain controller", "classic backup domain controller", "active
# directory domain controller".
#
# Most people will want "standalone server" or "member server".
# Running as "active directory domain controller" will require first
# running "samba-tool domain provision" to wipe databases and create a
# new domain.
server role = standalone server

# This option is important for security. It allows you to restrict
# connections to machines which are on your local network. The
```

In this case, we only put a couple of options that will allow us to ensure that the other machines can relate to it. That being said, the following configuration so that this machine is where we will put the files in the first instance.


```
[homes]
comment = carpetas por compartir
browseable = yes
writable = yes
path = /var/compartir
read only = no
guest ok = yes
create mask = 0777
directory mask = 0777
public = yes
```

Now we are going to start the Samba service on the system, unlike the previous machine, here we must give permissions to a series of files that could be said to be the ones that allow the package to start on the system. In addition, starting it manually based on the script created by samba.

```
enabled user samba.
root@slack:/var/compartir# chmod 777 /var/compartir
root@slack:/var/compartir# chmod -x /etc/rc.d/rc.samba
root@slack:/var/compartir# chmod 777 /etc/rc.d/rc.samba
root@slack:/var/compartir# /etc/rc.d/rc.samba start
Starting Samba:  /usr/sbin/smbd -D
                 /usr/sbin/nmbd -D
```

Finally, we only repeat the process that allows us to create a Samba user, with the difference that it is not necessary to have the user pre-created for the command to work. Thus, this machine in theory should be ready to work.

When we finally have all the machines configured, we proceed to make a connection between Slackware and NetBSD. To do this, both were turned on from different devices and it was confirmed that both worked correctly, including the internet. With the preparations already finished, we proceed to use the smbclient, which will use the SMB protocol to connect to a server that was activated in Slackware at the time of activating the package. For this client, you must indicate a series of parameters to make the connection. In the console we will write the command `'smbclient //10.2.77.27/compartir -U samba'` where: `'smbclient'` means the client; `'//10.2.77.27'` is the IP address of the machine hosting the server; `'/compartir'` is the folder or place intended for sharing; `'-U samba'` indicates the user who is getting inside the host server, this is a requirement to validate that they are really connecting. Now, we just send the command and wait. However, in our case an error appeared that we did not expect, it was the next one.

```
net# smbclient //10.2.77.27/compartir -U samba
Enter samba's password:
protocol negotiation failed: NT_STATUS_INVALID_NETWORK_RESPONSE
net# █
```

As we could see, we could not make the connection even though we had everything configured. Some of our theories; The thing is that the computers on which we carried out the test had some configuration that prevented us from establishing the connection. Another possibility is that some interference due to *VMware* affects the connection of packages in virtual machines. Anyway, we couldn't carry out the file transfer but we learned how the software worked well and it helped us understand how the SMB protocol works, despite not finding the connection error.

CONCLUTIONS

To conclude, in this laboratory we were able to use several software tools that allowed us to see how the network works internally and how it is distributed. In this way, we learned to identify common behaviors and situations in these. All this, through specialized courses that, together with autonomous research, gave us the necessary knowledge to meet the objectives of the guide and the lab. In addition, we were able to experiment with the networks and virtual machines that we created previously, communicating them through the same network or with packages or software specialized in file transfer and remote control in these operating systems. On the other hand, we were able to learn how to use code editors to manage files within the systems. In this way, it was possible to create scripts to adapt to the shell language and use it in future applications with networks and OS's.

VIDEOS

In this section you will find the videos made for this laboratory, these are:

- Summary and explanation of the "*Cisco Packet Tracer*" course.

<https://youtu.be/RiAX708cetA>

- Explanation of filters and tools within *Wireshark*.

<https://youtu.be/izH1v4gnNNY>

BIBLIOGRAFÍA

Bibliografía

Bash Reference Manual. (s/f). Gnu.org. Recuperado el 10 de febrero de 2025, de

<https://www.gnu.org/software/bash/manual/bash.html>

Capítulo 3. Uso de Samba como servidor. (s/f). Redhat.com. Recuperado el 10 de febrero de 2025, de

https://docs.redhat.com/es/documentation/red_hat_enterprise_linux/8/html/

[deploying_different_types_of_servers/assembly_using-samba-as-a-server_deploying-different-types-of-servers](#)

Cisco learning network. (s/f). Cisco.com. Recuperado el 10 de febrero de 2025, de

<https://learningnetwork.cisco.com/s/packet-tracer-alternative-lab-solutions>

¿Qué es Cisco Packet Tracer y cómo funciona? (2024, enero 2). Executrain.

<https://executrain.com.mx/que-es-cisco-packet-tracer-y-como-funciona/>

Setting up a Samba server on NetBSD. (2020, febrero 6). Dressupgeekout.com.

<https://dressupgeekout.com/netbsd/netbsd-samba/>

Singh, S. (2023, marzo 24). *How to communicate with two VMs in the same VNet*. DEV Community.

<https://dev.to/shishsingh/how-to-communicate-with-two-vms-in-the-same-vnet-1n6g>

Wireshark · documentation. (s/f). Wireshark. Recuperado el 10 de febrero de 2025, de

<https://www.wireshark.org/docs/>

Wireshark • learn. (s/f). Wireshark. Recuperado el 10 de febrero de 2025, de

<https://www.wireshark.org/learn>

(S/f). Samba.org. Recuperado el 10 de febrero de 2025, de <https://www.samba.org/>